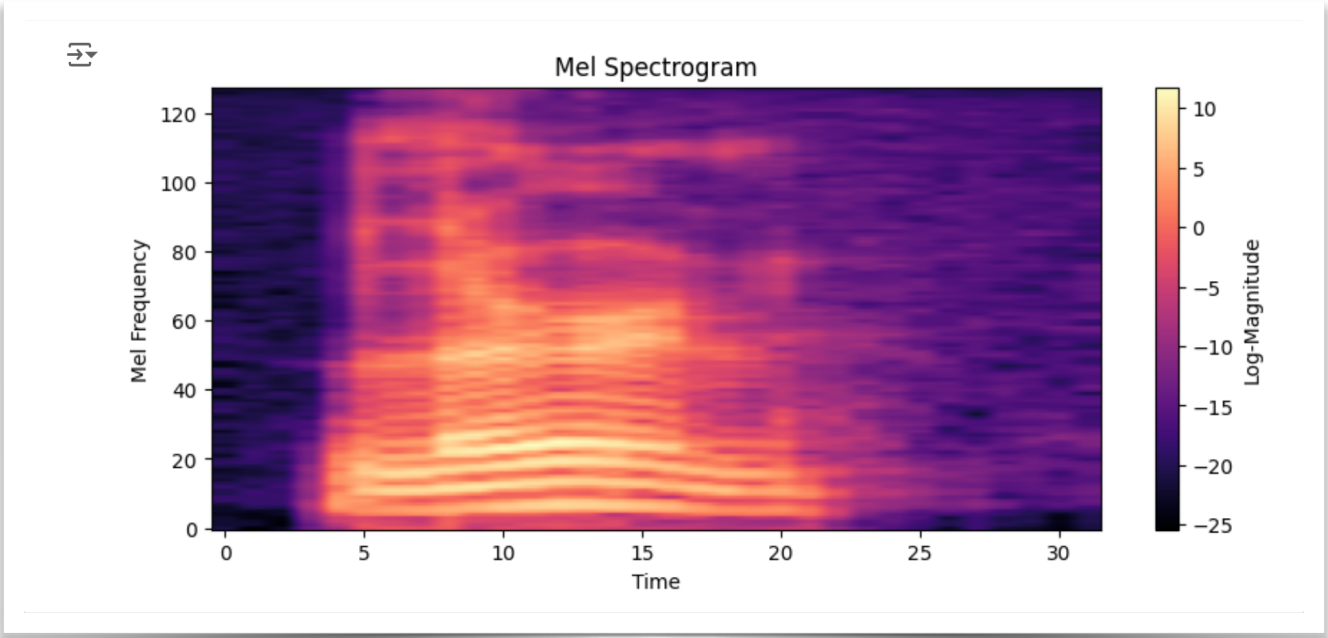
🎯 Aperçu du projet

Ce projet vise à reconnaître de courtes commandes vocales (ex : « yes », « no », « up », « right ») à partir de signaux audio bruts.

J'ai développé un pipeline complet de classification audio, allant du prétraitement des données à l'entraînement et à l'évaluation du modèle, en utilisant le dataset **SpeechCommands**.

Afin de rendre les données audio exploitables par un modèle de deep learning, chaque signal a été transformé en **spectrogramme Mel**, une représentation temps-fréquence du son.
Les zones claires correspondent aux fréquences dominantes, tandis que les zones sombres indiquent des signaux de faible intensité ou du silence.



⚙️ Méthodologie

📊 Jeu de données

- Dataset **SpeechCommands**
- 35 classes de commandes vocales
- Fichiers audio d'environ 1 seconde (16 kHz)

Prétraitement

- Conversion des signaux audio en **spectrogrammes Mel**
-

Paramètres utilisés :

- Fréquence d'échantillonnage : 16 000 Hz
- Taille de la fenêtre FFT : 1024
- Hop length : 512
- 128 bandes de fréquences Mel
-

Cette transformation permet au modèle de capturer à la fois les **dynamiques temporelles et fréquentielles** du signal audio.

🤖 Architecture du modèle

- Réseau de neurones convolutionnel **2D (CNN)**
- Couches de convolution + Batch Normalization
- MaxPooling + Adaptive Average Pooling
- Dropout pour la régularisation
- Couche fully connected en sortie (35 classes)

Le modèle contient environ **291 000 paramètres** et est spécifiquement conçu pour traiter des spectrogrammes comme des images 2D.

📈 Résultats

- ✅ Meilleure précision test : **~61 %**
- 📉 Apprentissage progressif mais instabilité entre les époques
- ⚠️ Confusion entre certaines classes proches (ex : « right » vs « up »)

Ces résultats mettent en évidence l'importance de :

- techniques de régularisation
- tuning des hyperparamètres
- équilibre du jeu de données

🔍 Points clés

- Les spectrogrammes améliorent fortement les performances par rapport à l'audio brut
- Les CNN 2D capturent efficacement les structures temps-fréquence
- Les performances sont sensibles à :
  - learning rate
  - distribution des données
  - stabilité de l'entraînement

💡 Applications

Ce type de modèle peut être utilisé pour :

- 🎤 Assistants vocaux
- 📱 Systèmes embarqués
- 🏭 Commande vocale en milieu industriel
- 🤖 Interaction homme-machine

🧠 Ce que j'ai appris

Ce projet m'a permis de :

- Construire un **pipeline complet de deep learning appliqué à l'audio**
- Comprendre le **traitement du signal avec les spectrogrammes Mel**
- Concevoir et entraîner un **CNN from scratch**
- Analyser les **performances et les limites d'un modèle**
- Identifier des problématiques comme le **surapprentissage** et la **confusion entre classes**